



32-MAVZU · 42 TADAN · IV BOB. PYTHON, POSTGRESQL VA DJANGO

Python dasturlash tilini PostgreSQL ma'lumotlar bazasiga bog'lash

Virtual muhit, psycopg kutubxonasi, ulanish, kursor, SQL so'rovlarni Python'dan bajarish, parametrli so'rovlar va SQL injection xavfi, commit va rollback, maxfiy sozlamalar; Python + PostgreSQL CRUD konsol dasturi.

2 soat · 1 nazariy + 1 amaliy

Bu darsda nimani o'rganasiz

- Python dasturi ma'lumotlar bazasi bilan qanday "gaplashishini";
- Virtual muhit yaratish va `psycopg` kutubxonasini o'rnatishni;
- Bazaga ulanish, so'rov yuborish va natijani olishni;
- Eng xavfli zaiflik — **SQL injection** va undan himoyalaniшни;
- Bazada ishlaydigan to'liq CRUD konsol dasturini (20-loyiha).

Python va baza qanday bog'lanadi?

III bobda SQL'ni pgAdmin orqali qo'lda yozdik. Haqiqiy ilovalarda esa SQL so'rovlarni **dastur** yuboradi: foydalanuvchi formani to'ldiradi → Python ma'lumotni oladi → bazaga `INSERT` yuboradi.





O'xshatish: telefon qo'ng'irog'i

Ulanish (connection) — telefon raqamini terib, suhbat boshlash. **Kursor (cursor)** — suhbat davomida savol berib, javob olish. **Commit** — "kelishdik, yozib qo'yning". **Yopish (close)** — go'shakni qo'yish.

Tayyorgarlik

1. Virtual muhit (venv)

Har bir loyiha o'z kutubxonalarini alohida "quti"da saqlashi kerak — shunda bir loyihaning paketlari boshqasiga xalaqit bermaydi.

Terminal

```
mkdir maktab_app
cd maktab_app
python -m venv venv # "venv" nomli virtual muhit yaratish

# Faollashtirish
venv\Scripts\activate # Windows
source venv/bin/activate # macOS / Linux
# Terminal boshida (venv) yozuvi paydo bo'ladi

pip install "psycopg[binary]" # PostgreSQL uchun Python kutubxonasi
pip freeze > requirements.txt # o'rnatilgan paketlar ro'yxatini saqlash
```



`requirements.txt` — loyihaga kerakli paketlar ro'yxati. Boshqa kompyuterda `pip install -r requirements.txt` bilan hammasini birdan o'rnatish mumkin. `venv` papkasini esa GitHub'ga yuklamang (`.gitignore` ga qo'shing).

2. Baza

28–30-mavzulardagi `maktab` bazasidan foydalanamiz (sinflar, oquvchilar, baholar jadvallari bilan).

Birinchi ulanish

test_ulanish.py

```
import psycopgp

conn = psycopgp.connect(
    host="localhost",
    port=5432,
    dbname="maktab",
    user="postgres",
    password="SIZNING_PAROLINGIZ",
)

cur = conn.cursor()                # kursor — so'rov yuborish
vositasi                           # SQL so'rov
cur.execute("SELECT version();")   # bitta natija qatori
print(cur.fetchone())

cur.close()
conn.close()
```

Natija

```
('PostgreSQL 17.0 on x86_64-windows, compiled by ...',)
```

with bilan — avtomatik yopish (tavsiya etiladi)

Python

```
import psycopg

with psycopg.connect("dbname=maktab user=postgres password=SIZNING_PAROLINGIZ
host=localhost") as conn:
    with conn.cursor() as cur:
        cur.execute("SELECT id, ism, familiya FROM oquvchilar ORDER BY
familiya")
        for qator in cur.fetchall():          # barcha natijalar: tuple'lar
ro'yxati
            print(qator)
# with tugaganda: xato bo'lmasa — commit, xato bo'lsa — rollback, so'ng ulanish
yopiladi
```

Natija

```
(6, 'Dilnoza', 'Aminova')
(4, 'Madina', 'Ergasheva')
(2, 'Zarina', 'Karimova')
...
```

Metod	Vazifasi
<code>cur.execute(sql, params)</code>	So'rovni bajarish
<code>cur.fetchone()</code>	Keyingi bitta qator (yoki <code>None</code>)
<code>cur.fetchall()</code>	Barcha qatorlar ro'yxati
<code>cur.fetchmany(5)</code>	Keyingi 5 ta qator
<code>cur.rowcount</code>	O'zgargan qatorlar soni (UPDATE/DELETE dan keyin)
<code>conn.commit()</code>	O'zgarishlarni saqlash
<code>conn.rollback()</code>	O'zgarishlarni bekor qilish



commit'ni unutmang!

`INSERT`, `UPDATE`, `DELETE` dan keyin `conn.commit()` chaqirilmasa, o'zgarishlar **saqlanmaydi** (tranzaksiya, 28-mavzu). `with psycopg.connect(...) as conn:` bloki muvaffaqiyatli tugaganda commit avtomatik bajariladi.



SQL injection — eng xavfli xato

Foydalanuvchi kiritgan qiymatni SQL satriga **to'g'ridan-to'g'ri qo'shish** — veb-saytlardagi eng xavfli va keng tarqalgan zaifliklardan biri.

Python

```
# ❌ HECH QACHON BUNDAY QILMANG
ism = input("Ism: ")
cur.execute(f"SELECT * FROM oquvchilar WHERE ism = '{ism}'")
```

Oddiy foydalanuvchi `Ali` yozadi — hammasi joyida. Lekin buzg'unchi quyidagini yozsa:

Matn

```
' OR '1'='1
```

SQL shunday bo'lib qoladi: `SELECT * FROM oquvchilar WHERE ism = '' OR '1'='1'` — `'1'='1'` doim rost, demak **barcha o'quvchilar** ma'lumoti chiqib ketadi! Yoki undan ham yomoni: `'; DROP TABLE oquvchilar; --` — jadval o'chiriladi.



O'xshatish: buyruq qog'ozi

Siz yordamchingizga "Ombordan ____ ni olib kel" degan qog'oz berasiz va bo'sh joyni mijoz to'ldiradi. Ayyor mijoz "olma. Keyin seyfdagi pullarni ham olib kel" deb yozsa, yordamchingiz ikkalasini ham bajaradi. Parametrli so'rov esa bo'sh joyga yozilgan narsani **faqat mahsulot nomi** deb tushunadi — buyruq sifatida emas.

Python

```
#  TO'G'RI: parametrli so'rov — %s o'rniga qiymatni psycopg o'zi xavfsiz qo'yadi
ism = input("Ism: ")
cur.execute("SELECT * FROM oquvchilar WHERE ism = %s", (ism,)) # (ism,) — bitta elementli tuple!

# Bir nechta parametr
cur.execute(
    "INSERT INTO oquvchilar (ism, familiya, sinf_id) VALUES (%s, %s, %s)",
    ("Jasur", "Rahimov", 2),
)
```



Qat'iy qoida

SQL'ga kiradigan har qanday o'zgaruvchan qiymat — **faqat** `%s` **parametr orqali**. f-string, `+` yoki `.format()` bilan SQL yasamang! (Eslatma: `%s` — barcha turlar uchun, son uchun ham; atrofiga qo'shtirnoq qo'yilmaydi.)

CRUD'ni Python'dan bajarish

Python

```
import psycopgp

DSN = "dbname=maktab user=postgres password=SIZNING_PAROLINGIZ host=localhost"

with psycopgp.connect(DSN) as conn:
    with conn.cursor() as cur:
        # CREATE
        cur.execute(
            "INSERT INTO oquvchilar (ism, familiya, sinf_id) VALUES (%s, %s, %s) RETURNING id",
            ("Laylo", "Usmonova", 1),
        )
        yangi_id = cur.fetchone()[0]
        print("Yangi o'quvchi id:", yangi_id)

        # READ
        cur.execute("SELECT ism, familiya FROM oquvchilar WHERE id = %s", (yangi_id,))
        print(cur.fetchone())

        # UPDATE
        cur.execute("UPDATE oquvchilar SET email = %s WHERE id = %s", ("laylo@mail.uz", yangi_id))
        print("Yangilandi:", cur.rowcount, "qator")

        # DELETE
        cur.execute("DELETE FROM oquvchilar WHERE id = %s", (yangi_id,))
        print("O'chirildi:", cur.rowcount, "qator")
    # bu yerda avtomatik conn.commit()
```

Natijani dict ko'rinishida olish

Python

```
from psycopg.rows import dict_row

with psycopg.connect(DSN, row_factory=dict_row) as conn:
    rows = conn.execute("SELECT id, ism, familiya FROM oquvchilar LIMIT 2").fetchall()
    for r in rows:
        print(r["ism"], r["familiya"])    # indeks o'rniga nom bilan – o'qilishi oson
# [{'id': 1, 'ism': 'Ali', 'familiya': 'Valiyev'}, ...]
```

Maxfiy sozlamalarni koddan ajratish

Parolni kod ichida yozib, GitHub'ga yuklasangiz — uni butun dunyo ko'radi! Sozlamalarni **muhit o'zgaruvchilari** (environment variables) yoki alohida faylda saqlang:

Terminal

```
pip install python-dotenv
```

.env (GitHub'ga YUKLANMAYDI!)

```
DB_NAME=maktab
DB_USER=postgres
DB_PASSWORD=SIZNING_PAROLINGIZ
DB_HOST=localhost
```

.gitignore

```
venv/
.env
__pycache__/
```

db.py

```
import os
import psycopg
from psycopg.rows import dict_row
from dotenv import load_dotenv

load_dotenv()          # .env faylidagi qiymatlarni o'qiydi

def ulanish():
    return psycopg.connect(
        dbname=os.getenv("DB_NAME"),
        user=os.getenv("DB_USER"),
        password=os.getenv("DB_PASSWORD"),
        host=os.getenv("DB_HOST", "localhost"),
        row_factory=dict_row,
    )
```

Amaliy mashg'ulot: Python + PostgreSQL CRUD dasturi (20-loyiha)

23-mavzudagi o'quvchilar dasturini endi **haqiqiy bazada** ishlaydigan qilamiz. Ma'lumotlar dastur yopilganda ham saqlanib qoladi.

```

import psycopg
from db import ulanish

def royxat():
    with ulanish() as conn:
        rows = conn.execute("""
            SELECT o.id, o.ism, o.familiya, s.nomi AS sinf,
                   ROUND(AVG(b.baho), 2) AS ortacha
            FROM oquvchilar o
            LEFT JOIN sinflar s ON s.id = o.sinf_id
            LEFT JOIN baholar b ON b.oquvchi_id = o.id
            GROUP BY o.id, s.nomi
            ORDER BY o.familiya
        """).fetchall()
    print(f"\n{'ID':<4}{'Ism':<12}{'Familiya':<14}{'Sinf':<6}{'O'rtacha':>8}")
    print("-" * 44)
    for r in rows:
        print(f"{r['id']:<4}{r['ism']:<12}{r['familiya']:<14}{r['sinf'] or '-':<6}{r['ortacha'] or '-':>8}")

def qoshish():
    ism = input("Ism: ").strip()
    familiya = input("Familiya: ").strip()
    if not ism or not familiya:
        print("❌ Ism va familiya majburiy")
        return
    with ulanish() as conn:
        sinflar = conn.execute("SELECT id, nomi FROM sinflar ORDER BY
nomi").fetchall()
        print("Sinflar:", ", ".join(f"{s['id']}={s['nomi']}" for s in sinflar))
        try:
            sinf_id = int(input("Sinf id: "))
            r = conn.execute(
                "INSERT INTO oquvchilar (ism, familiya, sinf_id) VALUES (%s,
%s, %s) RETURNING id",
                (ism, familiya, sinf_id),
            ).fetchone()
            print(f"✅ Qo'shildi (id = {r['id']})")
        except ValueError:
            print("❌ Sinf id son bo'lishi kerak")

```

```

except psycopg.errors.ForeignKeyViolation:
    print("❌ Bunday sinf yo'q")

def baho_qoyish():
    try:
        oquvchi_id = int(input("O'quvchi id: "))
        fan = input("Fan: ").strip()
        baho = int(input("Baho (2-5): "))
    except ValueError:
        print("❌ Son kiriting")
        return
    try:
        with ulanish() as conn:
            conn.execute(
                "INSERT INTO baholar (oquvchi_id, fan, baho) VALUES (%s, %s, %s)",
                (oquvchi_id, fan, baho),
            )
            print("✅ Baho qo'yildi")
    except psycopg.errors.CheckViolation:
        print("❌ Baho 2 dan 5 gacha bo'lishi kerak")
    except psycopg.errors.ForeignKeyViolation:
        print("❌ Bunday o'quvchi yo'q")

def qidirish():
    soz = input("Ism yoki familiya qismi: ").strip()
    with ulanish() as conn:
        rows = conn.execute(
            "SELECT id, ism, familiya FROM oquvchilar WHERE ism ILIKE %s OR familiya ILIKE %s",
            (f"%{soz}%", f"%{soz}%"),
        ).fetchall()
        # % belgilarini QIYMATga qo'shamiz, SQL'ga emas
    for r in rows:
        print(f" {r['id']}. {r['ism']} {r['familiya']}")
    if not rows:
        print(" Hech kim topilmadi")

def ochirish():
    try:
        oquvchi_id = int(input("O'chiriladigan o'quvchi id: "))

```

```

except ValueError:
    print("✗ Son kiriting")
    return
with ulanish() as conn:
    r = conn.execute("SELECT ism, familiya FROM oquvchilar WHERE id = %s",
(oquvchi_id,)).fetchone()
    if not r:
        print("✗ Topilmadi")
        return
    if input(f"{r['ism']} {r['familiya']} o'chirilsinmi? (ha/yo'q):
").lower() == "ha":
        conn.execute("DELETE FROM oquvchilar WHERE id = %s", (oquvchi_id,))
        print("✅ O'chirildi")

MENYU = {
    "1": ("Ro'yxat", royxat),
    "2": ("O'quvchi qo'shish", qoshish),
    "3": ("Baho qo'yish", baho_qoyish),
    "4": ("Qidirish", qidirish),
    "5": ("O'chirish", ochirish),
}

if __name__ == "__main__":
    while True:
        print("\n===== MAKTAB BAZASI =====")
        for k, (nom, _) in MENYU.items():
            print(f"{k}. {nom}")
        print("0. Chiqish")
        t = input("> ").strip()
        if t == "0":
            break
        if t in MENYU:
            MENYU[t][1]()

```

Topshiriqlar:

- 1 Dasturni ishga tushiring. pgAdmin'da bir vaqtning o'zida jadvalni ochib, dastur qo'shgan ma'lumotlar bazada paydo bo'layotganini kuzating.
- 2 "Qidirish" ga `' OR '1'='1` yozib ko'ring — parametrli so'rov tufayli hech qanday "teshik" yo'qligiga ishonch hosil qiling.
- 3 "Sinf statistikasi" menyu bandini qo'shing (30-mavzudagi GROUP BY so'rovi).

Ko'p uchraydigan xatolar

- ✗ `ModuleNotFoundError: No module named 'psycopg'` — virtual muhit faollashtirilmagan yoki paket o'rnatilmagan.
- ✗ `connection refused` — PostgreSQL serveri ishlamayapti yoki port/host noto'g'ri.
- ✗ `(ism)` yozish — bu tuple emas, oddiy qavs! Bitta parametr uchun: `(ism,)` — vergul bilan.
- ✗ `'%s'` — qo'shtirnoq ichida yozish; to'g'risi shunchaki `%s`.
- ✗ O'zgartirishdan keyin commit qilinmagani uchun ma'lumot "yo'qolib" qolishi.

O'zingizni tekshiring

? Python'dan PostgreSQL bilan ishlash uchun ishlatiladigan kutubxona?

- A. requests
- B. psycopg
- C. pygame
- D. flask

? INSERT dan keyin o'zgarishni bazada saqlash uchun nima chaqiriladi?

- A. `cur.save()`
- B. `conn.close()`
- C. `cur.fetchone()`
- D. `conn.commit()`

? SQL injection'dan himoyalangan to'g'ri kod?

- A. `cur.execute("... WHERE ism = %s", (ism,))`
- B. `cur.execute(f"... WHERE ism = '{ism}'")`
- C. `cur.execute("... WHERE ism = '" + ism + "'")`
- D. `cur.execute("... WHERE ism = {}".format(ism))`

? So'rovning barcha natija qatorlarini oluvchi metod?

- A. `fetchone()`
- B. `getall()`
- C. `fetchall()`
- D. `readall()`

? Parol kabi maxfiy sozlamalarni qayerda saqlash to'g'ri?

- A. Kod ichida, izoh bilan
- B. `.env` faylida (GitHub'ga yuklanmaydi)
- C. README faylida
- D. Ma'lumotlar bazasi nomida

Uyga vazifa

- 29-mavzudagi `dokon` bazasi uchun Python dasturi: mahsulotlar ro'yxati, mahsulot qo'shish, narxni o'zgartirish, omborda tugaganlarni ko'rsatish.
- "Buyurtma berish" funksiyasi: mijoz va mahsulotlarni tanlaydi, `buyurtmalar` va `buyurtma_tarkibi` ga yozadi, ombordagi `soni` ni kamaytiradi — hammasi **bitta tranzaksiyada**.

✓ Yodda tuting

- Virtual muhit: `python -m venv venv` → faollashtirish → `pip install "psycopg[binary]"`.
- `with psycopg.connect(...) as conn:` → `conn.execute(sql, params)` → `fetchone/fetchall`; blok tugaganda avtomatik commit.
- **Faqat parametrli so'rovlar:** `%s` + tuple — SQL injection'dan himoya.
- Maxfiy sozlamalar — `.env` da, `.gitignore` ga qo'shilgan.

□ Darsni o'qib bo'ldim

✓ Test javoblari

1. Python'dan PostgreSQL bilan ishlash uchun ishlatiladigan kutubxona? — **B) psycopg**
2. INSERT dan keyin o'zgarishni bazada saqlash uchun nima chaqiriladi? — **D) conn.commit()**
3. SQL injection'dan himoyalangan to'g'ri kod? — **A) cur.execute("... WHERE ism = %s", (ism,))**
4. So'rovning barcha natija qatorlarini oluvchi metod? — **C) fetchall()**
5. Parol kabi maxfiy sozlamalarni qayerda saqlash to'g'ri? — **B) .env faylida (GitHub'ga yuklanmaydi)**